ESTRUTURA DO SISTEMA

📦 Estrutura do Projeto (clean):

```
├── backend
│   ├── app
│   │   ├── auth
│   │   │   ├── __init__.py
│   │   │   └── auth_service.py
│   │   ├── services
│   │   │   ├── __init__.py
│   │   │   └── rag_service.py
│   │   ├── __init__.py
│   │   └── database.py
│   ├── docs
│   │   ├── BIA_RAG.docx
│   │   └── BIA_RAG.pdf
│   ├── criar_admin.py
│   ├── criar_usuario_admin.py
│   ├── main.py
│   └── requirements.txt
├── frontend
│   ├── assets
│   │   ├── bg_hero.png
│   │   ├── bg_hero_old.png
│   │   ├── favicon.ico
│   │   ├── Hub.jpeg
│   │   ├── hub01.jpeg
│   │   └── ImagemPrincipal.png
│   ├── ui
│   │   ├── pages
```

```
│ │ │  ├── admin_rag.py
│ │ │  ├── consultoria.py
│ │ │  ├── dashboard.py
│ │ │  ├── hub.py
│ │ │  ├── index.py
│ │ │  └── login.py
│ │  ├── __init__.py
│ │  ├── state.py
│ │  ├── styles.py
│ │  └── ui.py
│  ├── .gitignore
│  ├── requirements.txt
│  └── rxconfig.py
├── .gitignore
├── AnaliseClaude260331.pdf
├── Dockerfile
├── Estrutura_dir.py
├── Supabase
└── xxx.env
```

ARQUIVOS COM OS CÓDIGOS DATA 02/04/26

Vou fornecer o nome do arquivo e em seguida o código completo

backend/requirements.txt

```
alembic==1.18.4
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.13.0
bcrypt==4.0.1
bidict==0.23.1
cachetools==6.2.6
certifi==2026.2.25
cffi==2.0.0
charset-normalizer==3.4.6
click==8.3.1
colorama==0.4.6
cryptography==46.0.5
deprecation==2.1.0
dnspython==2.8.0
email-validator==2.3.0
fastapi==0.135.2
fastapi-cli==0.0.24
fastapi-cloud-cli==0.15.0
fastar==0.9.0
fsspec==2026.2.0
granian==2.7.2
greenlet==3.3.2
h11==0.16.0
h2==4.3.0
hpack==4.1.0
httpcore==1.0.9
httptools==0.7.1
httpx==0.28.1
hyperframe==6.1.0
idna==3.11
itsdangerous==2.2.0
Jinja2==3.1.6
Mako==1.3.10
markdown-it-py==4.0.0
MarkupSafe==3.0.3
mdurl==0.1.2
mmh3==5.2.1
multidict==6.7.1
passlib==1.7.4
platformdirs==4.9.4
postgrest==2.28.3
propcache==0.4.1
psutil==7.2.2
pycparser==3.0
```

pydantic==2.12.5
pydantic-extra-types==2.11.1
pydantic-settings==2.13.1
pydantic_core==2.41.5
Pygments==2.19.2
pyiceberg==0.11.1
PyJWT==2.12.1
pyparsing==3.3.2
pyroaring==1.0.4
python-dateutil==2.9.0.post0
python-dotenv==1.2.2
python-engineio==4.13.1
python-multipart==0.0.22
python-socketio==5.16.1
PyYAML==6.0.3
realtime==2.28.3
redis==7.4.0
reflex==0.8.28.post1
reflex-hosting-cli==0.1.62
requests==2.32.5
rich==14.3.3
rich-toolkit==0.19.7
rignore==0.7.6
sentry-sdk==2.56.0
setuptools==80.10.2
shellingham==1.5.4
simple-websocket==1.1.0
six==1.17.0
SQLAlchemy==2.0.48
sqlmodel==0.0.37
starlette==1.0.0
storage3==2.28.3
StrEnum==0.4.15
strictyaml==1.7.3
supabase==2.28.3
supabase-auth==2.28.3
supabase-functions==2.28.3
tenacity==9.1.4
typer==0.24.1
typing-inspection==0.4.2
typing_extensions==4.15.0
urllib3==2.6.3
uvicorn==0.42.0
watchfiles==1.1.1
websockets==15.0.1
wheel==0.46.3
wrapt==2.1.2
wsproto==1.3.2

```
yarl==1.23.0
zstandard==0.23.0
langchain==0.3.25
langchain-chroma==0.2.4
langchain-community==0.3.24
langchain-core==0.3.64
langchain-openai==0.3.19
langchain-text-splitters==0.3.8
langsmith==0.3.45
faiss-cpu==1.8.0
pypdf==4.1.0
tiktoken==0.7.0
```

backend/main.py

```
from fastapi import FastAPI, HTTPException, Request, Depends, status
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import HTTPBearer
from pydantic import BaseModel
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
import logging
import os
import jwt

# Importações de módulos locais
from app.auth.auth_service import (
    verify_password,
    create_access_token,
    validar_email,
    SECRET_KEY,
    ALGORITHM
)
from app.database import supabase
from app.services.rag_service import rag_service

# --- Configuração de Logging ---
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s
- %(message)s')
logger = logging.getLogger(__name__)

# --- Inicialização do FastAPI ---
app = FastAPI(
    title="Dr. Tilápia 1.3 API",
    description="API para o sistema de monitoramento e consultoria de piscicultura Dr.
Tilápia.",
    version="1.3.0"
)

# --- Configuração CORS Restritiva ---
allowed_origins = [
    "http://localhost:3000",
    "http://127.0.0.1:3000",
    "http://localhost:8000",
    "http://127.0.0.1:8000",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=allowed_origins,
```

```
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# --- Configuração de Rate Limiting ---
limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)


# --- Modelos Pydantic para Requisições ---
class LoginRequest(BaseModel):
    """Modelo para a requisição de login."""
    email: str
    password: str


class ChatRequest(BaseModel):
    """Modelo para a requisição de chat RAG."""
    message: str
    history: list = []


# --- Segurança JWT ---
security = HTTPBearer()


def verify_token(credentials=Depends(security)):
    """
    Verifica a validade de um token JWT fornecido no cabeçalho Authorization.

    Args:
        credentials: Credenciais HTTP do tipo Bearer.

    Returns:
        dict: O payload do token se for válido.

    Raises:
        HTTPException: Se o token for inválido ou expirado.
    """
    token = credentials.credentials
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except jwt.ExpiredSignatureError:
        logger.warning("Tentativa de acesso com token expirado.")
        raise HTTPException(
```

```
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Token de autenticação expirado",
        headers={"WWW-Authenticate": "Bearer"},
    )
  except jwt.InvalidTokenError:
    logger.warning("Tentativa de acesso com token inválido.")
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Token de autenticação inválido",
        headers={"WWW-Authenticate": "Bearer"},
    )
  except Exception as e:
    logger.error(f"Erro inesperado na verificação de token: {e}")
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Não foi possível validar as credenciais",
        headers={"WWW-Authenticate": "Bearer"},
    )


# --- Endpoints da API ---

@app.get("/health", summary="Verifica a saúde da API")
async def health_check():
  """
  Endpoint simples para verificar se a API está online e respondendo.
  """
  logger.info("Requisição /health recebida.")
  return {"status": "healthy", "version": app.version}


@app.post("/auth/login", summary="Autentica um usuário e retorna um token JWT")
@limiter.limit("5/minute")
async def login(request: Request, credentials: LoginRequest):
  """
  Processa a autenticação do usuário.

  Args:
    request (Request): Objeto da requisição FastAPI (usado pelo rate limiter).
    credentials (LoginRequest): Email e senha do usuário.

  Returns:
    dict: Um token de acesso JWT e informações básicas do usuário.

  Raises:
    HTTPException: Se as credenciais forem inválidas, email malformado ou erro interno.
  """
  logger.info(f"Tentativa de login para o email: {credentials.email}")
```

```
    try:
        # 1. Validação do formato do email
        if not validar_email(credentials.email):
            logger.warning(f"Login falhou: Email inválido '{credentials.email}'.")
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Formato de email inválido."
            )

        # 2. Buscar usuário no Supabase
        user_response = supabase.table("users").select("*").eq("email",
credentials.email).execute()
        user_data = user_response.data

        if not user_data:
            logger.warning(f"Login falhou: Usuário não encontrado para '{credentials.email}'.")
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Credenciais inválidas."
            )

        user = user_data[0]

        # 3. Verificar senha
        if not verify_password(credentials.password, user.get("password_hash", "")):
            logger.warning(f"Login falhou: Senha incorreta para '{credentials.email}'.")
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Credenciais inválidas."
            )

        # 4. Gerar token JWT
        access_token = create_access_token(data={"sub": credentials.email})

        logger.info(f"Login bem-sucedido para '{credentials.email}'.")
        return {
            "status": "success",
            "access_token": access_token,
            "token_type": "bearer",
            "user_email": credentials.email,
            "user_name": user.get("name", credentials.email.split("@")[0])
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.error(f"Erro inesperado durante o login para '{credentials.email}': {e}",
exc_info=True)
        raise HTTPException(
```

```
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail="Erro interno do servidor ao processar o login."
    )


@app.post("/consultoria/chat", summary="Envia uma mensagem para o serviço de
consultoria IA (RAG)")
async def chat_consultoria(request: ChatRequest, token_payload: dict =
Depends(verify_token)):
    """
    Processa uma requisição de chat com o serviço RAG. Requer autenticação JWT.

    Args:
        request (ChatRequest): Mensagem do usuário e histórico da conversa.
        token_payload (dict): Payload do token JWT verificado (injetado por Depends).

    Returns:
        dict: A resposta gerada pelo modelo RAG.

    Raises:
        HTTPException: Se a autenticação falhar ou ocorrer um erro no serviço RAG.
    """
    user_email = token_payload.get("sub", "desconhecido")
    logger.info(f"Requisição de chat de '{user_email}': '{request.message[:50]}...'")
    try:
        formatted_history = [tuple(item) for item in request.history]
        response = await rag_service.get_response(request.message, formatted_history)

        logger.info(f"Resposta RAG gerada para '{user_email}'.")
        return {"answer": response}
    except Exception as e:
        logger.error(f"Erro no serviço RAG para '{user_email}': {e}", exc_info=True)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail="Erro ao processar a consulta com a IA."
        )


@app.get("/metrics/latest", summary="Retorna as métricas mais recentes do sistema")
async def get_metrics(token_payload: dict = Depends(verify_token)):
    """
    Retorna as métricas mais recentes do sistema de monitoramento. Requer autenticação
JWT.

    Args:
        token_payload (dict): Payload do token JWT verificado (injetado por Depends).

    Returns:
```

dict: Dicionário contendo as métricas (temperatura, oxigênio, pH, amônia).

Raises:
HTTPException: Se a autenticação falhar ou ocorrer um erro na obtenção das métricas.

```
    """
    user_email = token_payload.get("sub", "desconhecido")
    logger.info(f"Requisição de métricas de '{user_email}'.")
    try:
        metrics_data = {
            "temperatura": 26.5,
            "oxigenio": 7.2,
            "ph": 6.8,
            "amonia": 0.5
        }
        logger.info(f"Métricas retornadas para '{user_email}'.")
        return metrics_data
    except Exception as e:
        logger.error(f"Erro ao obter métricas para '{user_email}': {e}", exc_info=True)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail="Erro ao buscar as métricas do sistema."
        )
```

backend/criar_usuario_admin.py

```
import os
import logging
import re
from supabase import create_client, Client
from app.services.user_service import UserService
from dotenv import load_dotenv
from typing import List, Dict, Any, Optional

# Carrega variáveis de ambiente do arquivo .env
load_dotenv()

# Configuração de logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s
- %(message)s')
logger = logging.getLogger(__name__)


class AdminCLI:
    """
    Interface de linha de comando para gerenciar usuários do Supabase Auth.
    Utiliza o UserService para operações administrativas.
    """

    def __init__(self):
        try:
            self.user_service = UserService()
            logger.info("AdminCLI inicializado com UserService.")
        except ValueError as e:
            logger.critical(f"Erro de inicialização do UserService: {e}")
            print(f"\n❌ ERRO CRÍTICO: {e}")
            print(
                "   Certifique-se de que SUPABASE_URL e SUPABASE_SERVICE_ROLE_KEY estão
definidos no seu arquivo .env.")
            exit(1)

    def _validate_email(self, email: str) -> bool:
        """Valida o formato do email."""
        if not re.fullmatch(r"[^@]+@[^@]+\.[^@]+", email):
            print("❌ Formato de email inválido.")
            return False
        return True

    def _validate_password_strength(self, password: str) -> bool:
        """Valida a força da senha (mínimo 8 caracteres, maiúscula, minúscula, número,
caractere especial)."""
        if len(password) < 8:
```

```
        print("❌ A senha deve ter no mínimo 8 caracteres.")
        return False
    if not re.search(r"[A-Z]", password):
        print("❌ A senha deve conter pelo menos uma letra maiúscula.")
        return False
    if not re.search(r"[a-z]", password):
        print("❌ A senha deve conter pelo menos uma letra minúscula.")
        return False
    if not re.search(r"\d", password):
        print("❌ A senha deve conter pelo menos um número.")
        return False
    if not re.search(r"[!@#$%^&*(),.?\"{}\|<>]", password):
        print("❌ A senha deve conter pelo menos um caractere especial.")
        return False
    return True

def _get_user_input(self, prompt: str, sensitive: bool = False) -> str:
    """Obtém entrada do usuário, com opção para entrada sensível (senha)."""
    if sensitive:
        return input(prompt)
    return input(prompt)

def display_menu(self):
    """Exibe o menu de opções para o usuário."""
    print("\n--- Gerenciamento de Usuários Supabase ---")
    print("1. Criar Novo Usuário")
    print("2. Listar Usuários Existentes")
    print("3. Deletar Usuário")
    print("4. Sair")
    print("")

def handle_create_user(self):
    """Processa a criação de um novo usuário."""
    print("\n--- Criar Novo Usuário ---")
    while True:
        email = self._get_user_input("Email do novo usuário: ").strip()
        if self._validate_email(email):
            break

    existing_user = self.user_service.get_user_by_email(email)
    if existing_user:
        print(f"⚠️ Usuário com email '{email}' já existe. ID: {existing_user.get('id')}")
        return

    while True:
        password = self._get_user_input("Senha do novo usuário: ", sensitive=True).strip()
        if self._validate_password_strength(password):
```

```
            break

        name = self._get_user_input("Nome completo do usuário: ").strip()

        roles = ["admin", "user", "guest"]
        while True:
            role = self._get_user_input(f"Role do usuário ({', '.join(roles)}): ").strip().lower()
            if role in roles:
                break
            print(f"❌ Role inválida. Escolha entre: {', '.join(roles)}")

        try:
            new_user = self.user_service.create_user(email, password, name, role=role)
            print(f"\n🎉 Usuário '{email}' criado com sucesso!")
            print(f"   ID: {new_user.get('id')}")
            print(f"   Nome: {new_user.get('user_metadata', {}).get('name')}")
            print(f"   Email: {new_user.get('email')}")
            print(f"   Role: {new_user.get('user_metadata', {}).get('role')}")
        except Exception as e:
            logger.error(f"Erro ao criar usuário '{email}': {e}", exc_info=True)
            print(f"\n❌ ERRO ao criar usuário: {e}")

    def handle_list_users(self):
        """Processa a listagem de usuários existentes."""
        print("\n--- Listar Usuários ---")
        try:
            users = self.user_service.list_users()
            if not users:
                print("Nenhum usuário encontrado.")
                return

            print(f"Total de usuários: {len(users)}")
            for user in users:
                user_id = user.get('id', 'N/A')
                email = user.get('email', 'N/A')
                name = user.get('user_metadata', {}).get('name', 'N/A')
                role = user.get('user_metadata', {}).get('role', 'N/A')
                created_at = user.get('created_at', 'N/A')
                print(f"")
                print(f"ID: {user_id}")
                print(f"Email: {email}")
                print(f"Nome: {name}")
                print(f"Role: {role}")
                print(f"Criado em: {created_at}")
            print(f"")
        except Exception as e:
            logger.error(f"Erro ao listar usuários: {e}", exc_info=True)
```

```
        print(f"\n❌ ERRO ao listar usuários: {e}")

    def handle_delete_user(self):
        """Processa a exclusão de um usuário."""
        print("\n--- Deletar Usuário ---")
        user_id = self._get_user_input("ID do usuário a ser deletado: ").strip()
        if not user_id:
            print("❌ ID do usuário não pode ser vazio.")
            return

        try:
            user_to_delete = self.user_service.get_user_by_id(user_id)
            if not user_to_delete:
                print(f"⚠️ Usuário com ID '{user_id}' não encontrado.")
                return

            confirm = self._get_user_input(
                f"Tem certeza que deseja deletar o usuário '{user_to_delete.get('email')}' (s/N)?
").strip().lower()
            if confirm == 's':
                self.user_service.delete_user(user_id)
                print(f"\n✅ Usuário '{user_id}' deletado com sucesso.")
            else:
                print("Operação de exclusão cancelada.")
        except Exception as e:
            logger.error(f"Erro ao deletar usuário '{user_id}': {e}", exc_info=True)
            print(f"\n❌ ERRO ao deletar usuário: {e}")

    def run(self):
        """Executa o loop principal da interface de linha de comando."""
        while True:
            self.display_menu()
            choice = self._get_user_input("Escolha uma opção: ").strip()

            if choice == '1':
                self.handle_create_user()
            elif choice == '2':
                self.handle_list_users()
            elif choice == '3':
                self.handle_delete_user()
            elif choice == '4':
                print("Saindo do gerenciador de usuários. Até mais!")
                break
            else:
                print("Opção inválida. Por favor, tente novamente.")
```

```
if __name__ == "__main__":
    cli = AdminCLI()
    cli.run()
```

backend/.env

SUPABASE_URL=https://cfupucrcnyqpimrsmbws.supabase.co
SUPABASE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImNmdXB1Y3JjbnlxcGltcnNtYndzIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NzQzOTIwMjYsImV4cCI6MjA4OTk2ODAyNn0.2QUqEUb5mxwKZboRKpJUyaR7bDuTemekzvYAFgeLWRs
SECRET_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImNmdXB1Y3JjbnlxcGltcnNtYndzIiwicm9sZSI6InNlcnZpY2Vfcm9sZSIsImlhdCI6MTc3NDM5MjAyNiwiZXhwIjoyMDg5OTY4MDI2fQ.0wXSbrOI4SSdTIPDbKXQttADOTTEq5gqUvM8oqXIlfs
OPENAI_API_KEY=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

backend/app/__init__.py

```
"""
Pacote principal da aplicação DrTilápia 1.3.
Centraliza importações de módulos críticos.
"""

__version__ = "2.0"
```

backend/app/database.py

```
import os
from pathlib import Path
from supabase import create_client, Client
from dotenv import load_dotenv

# Configuração de caminhos para evitar problemas com OneDrive/Windows
basedir = Path(__file__).resolve().parent.parent
env_path = basedir / '.env'
load_dotenv(dotenv_path=env_path)

url: str = os.environ.get("SUPABASE_URL")
key: str = os.environ.get("SUPABASE_KEY")

if not url or not key:
    # Fallback para diretório atual
    load_dotenv()
    url = os.environ.get("SUPABASE_URL")
    key = os.environ.get("SUPABASE_KEY")

# Objeto global do Supabase utilizado por todo o sistema
supabase: Client = create_client(url, key)
```

backend/app/services/__init__.py

“”

backend/app/services/rag_service.py

```
import os
import logging
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_community.document_loaders import PyPDFLoader, DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.chains import ConversationalRetrievalChain

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("RAGService")


class RAGService:
    def __init__(self):
        self.embeddings = OpenAIEmbeddings()
        self.vectorstore_path = "vectorstore_faiss"
        self.docs_path = "docs/"
        self._vectorstore = None

    def _load_vectorstore(self):
        if os.path.exists(os.path.join(self.vectorstore_path, "index.faiss")):
            return FAISS.load_local(self.vectorstore_path, self.embeddings,
allow_dangerous_deserialization=True)

        if not os.path.exists(self.docs_path): os.makedirs(self.docs_path)
        loader = DirectoryLoader(self.docs_path, glob="*.pdf", loader_cls=PyPDFLoader)
        documents = loader.load()
        if not documents: return None

        text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
        chunks = text_splitter.split_documents(documents)
        vectorstore = FAISS.from_documents(chunks, self.embeddings)
        vectorstore.save_local(self.vectorstore_path)
        return vectorstore

    async def get_response(self, question, chat_history):
        if self._vectorstore is None: self._vectorstore = self._load_vectorstore()
        if self._vectorstore is None: return {"answer": "Adicione PDFs em /docs.", "sources": []}

        chain = ConversationalRetrievalChain.from_llm(
            llm=ChatOpenAI(model="gpt-4-turbo", temperature=0),
            retriever=self._vectorstore.as_retriever(search_kwargs={"k": 3}),
            return_source_documents=True
        )
        result = chain.invoke({"question": question, "chat_history": chat_history})
        return {"answer": result["answer"],
```

```
        "sources": [doc.metadata.get("source") for doc in result["source_documents"]]}

rag_service = RAGService()
```

backend/app/auth/__init__.py

“”

backend/app/auth/auth_service.py

```
import jwt
import os
from datetime import datetime, timedelta
from passlib.context import CryptContext
from email_validator import validate_email, EmailNotValidError
import logging
from typing import Optional

# Configuração de Logging
logger = logging.getLogger(__name__)

# --- Configurações de Segurança ---
SECRET_KEY = os.environ.get("SECRET_KEY",
"uma_chave_secreta_muito_forte_e_aleatoria_para_o_DrTilapia_1.3")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_HOURS = 24

# Contexto para hashing de senhas (bcrypt)
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")


# --- Funções de Validação ---

def validar_email(email: str) -> bool:
    """
    Valida o formato de um endereço de e-mail usando a biblioteca email_validator.

    Args:
        email (str): O endereço de e-mail a ser validado.

    Returns:
        bool: True se o e-mail for válido, False caso contrário.
    """
    try:
        validate_email(email)
        return True
    except EmailNotValidError as e:
        logger.warning(f"Validação de e-mail falhou para '{email}': {e}")
        return False
    except Exception as e:
        logger.error(f"Erro inesperado na validação de e-mail para '{email}': {e}")
        return False


def validar_senha(senha: str) -> bool:
    """
```

```
    Valida a força de uma senha com base em critérios mínimos.

    Critérios:
    - Mínimo de 8 caracteres.
    - Pelo menos uma letra maiúscula.
    - Pelo menos um dígito.
    - Pelo menos um caractere especial.

    Args:
        senha (str): A senha a ser validada.

    Returns:
        bool: True se a senha atender aos critérios de força, False caso contrário.
    """
    if len(senha) < 8:
        logger.warning("Validação de senha falhou: Senha muito curta (mínimo 8 caracteres).")
        return False
    if not any(c.isupper() for c in senha):
        logger.warning("Validação de senha falhou: Senha deve conter pelo menos uma letra maiúscula.")
        return False
    if not any(c.isdigit() for c in senha):
        logger.warning("Validação de senha falhou: Senha deve conter pelo menos um dígito.")
        return False

    special_characters = "!@#$%^&*()_+-=[]{}|;:,.<>?"
    if not any(c in special_characters for c in senha):
        logger.warning("Validação de senha falhou: Senha deve conter pelo menos um caractere especial.")
        return False
    return True


# --- Funções de Hashing de Senha ---

def get_password_hash(password: str) -> str:
    """
    Gera o hash de uma senha usando bcrypt.

    Args:
        password (str): A senha em texto puro.

    Returns:
        str: O hash da senha.
    """
    return pwd_context.hash(password)
```

```
def verify_password(plain_password: str, hashed_password: str) -> bool:
    """
    Verifica se uma senha em texto puro corresponde a um hash de senha.

    Args:
        plain_password (str): A senha em texto puro fornecida pelo usuário.
        hashed_password (str): O hash da senha armazenado.

    Returns:
        bool: True se as senhas corresponderem, False caso contrário.
    """
    try:
        return pwd_context.verify(plain_password, hashed_password)
    except Exception as e:
        logger.error(f"Erro ao verificar senha: {e}")
        return False


# --- Funções JWT (JSON Web Token) ---

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    """
    Cria um token de acesso JWT.

    Args:
        data (dict): Dados a serem codificados no token (ex: {"sub": "user@example.com"}).
        expires_delta (Optional[timedelta]): Tempo de expiração do token. Se None, usa o
padrão.

    Returns:
        str: O token JWT codificado.
    """
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(hours=ACCESS_TOKEN_EXPIRE_HOURS)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    logger.info(f"Token JWT criado para dados: {data.get('sub', 'N/A')}")
    return encoded_jwt


def verify_access_token(token: str) -> Optional[dict]:
    """
    Verifica e decodifica um token de acesso JWT.
```

```
    Args:
        token (str): O token JWT a ser verificado.

    Returns:
        Optional[dict]: O payload do token se for válido, None caso contrário.
    """
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        logger.info(f"Token JWT verificado com sucesso. Payload: {payload.get('sub', 'N/A')}")
        return payload
    except jwt.ExpiredSignatureError:
        logger.warning("Verificação de token falhou: Token expirado.")
        return None
    except jwt.InvalidTokenError:
        logger.warning("Verificação de token falhou: Token inválido.")
        return None
    except Exception as e:
        logger.error(f"Erro inesperado na verificação de token: {e}")
        return None
```

frontend/rxconfig.py

```
import reflex as rx

config = rx.Config(
    app_name="ui",
    # 🌐 Porta onde o motor interno do Reflex vai rodar (Websocket/State)
    backend_port=8005,
    # 🔗 URL que o Frontend usará para falar com o motor do Reflex
    backend_url="http://localhost:8002",
)
```

frontend/requirements.txt

reflex==0.8.28.post1

frontend/.gitignore

```
.web
__pycache__/
.states
*.py[cod]
assets/external/
*.db
```

frontend/ui/__init__.py

“”

frontend/ui/state.py

```
import reflex as rx
import httpx
import logging
from typing import List, Tuple

logger = logging.getLogger(__name__)
BACKEND_URL = "http://localhost:8080"


class State(rx.State):
    """Estado global do DrTilápia 1.3."""

    # --- AUTENTICAÇÃO ---
    user_email: str = ""
    user_name: str = ""
    access_token: str = ""
    is_authenticated: bool = False

    # --- CHAT RAG ---
    current_message: str = ""
    chat_history: List[Tuple[str, str]] = []
    is_loading: bool = False
    error_message: str = ""

    # --- MÉTRICAS ---
    temperatura: float = 26.5
    oxigenio: float = 7.2
    ph: float = 6.8
    amonia: float = 0.5
    loading_metrics: bool = False

    # --- SETTERS ---
    def set_user_email(self, email: str):
        self.user_email = email

    def set_current_message(self, msg: str):
        self.current_message = msg

    def set_password(self, password: str):
        self.current_message = password

    # --- PROTEÇÃO DE ROTAS ---
    def check_login(self):
        """Redireciona se não autenticado."""
        if not self.is_authenticated:
            return rx.redirect("/login")
```

```
def logout(self):
    """Realiza logout."""
    self.user_email = ""
    self.user_name = ""
    self.access_token = ""
    self.is_authenticated = False
    self.chat_history = []
    self.error_message = ""
    self.current_message = ""
    return rx.redirect("/login")

# --- LOGIN ---
async def handle_login(self):
    """Realiza login com validação."""
    if not self.user_email or not self.current_message:
        self.error_message = "Preencha todos os campos."
        return

    self.is_loading = True
    try:
        async with httpx.AsyncClient() as client:
            res = await client.post(
                f"{BACKEND_URL}/auth/login",
                json={"email": self.user_email, "password": self.current_message},
                timeout=10.0
            )

            if res.status_code == 200:
                data = res.json()
                # CRÍTICO: Atualizar estado ANTES de redirecionar
                self.access_token = data.get("access_token", "")
                self.user_name = data.get("user_name", "")
                self.is_authenticated = True  # Marcar como autenticado
                self.error_message = ""
                self.current_message = ""
                logger.info(f"Login bem-sucedido para: {self.user_email}")
                # Redirecionar após atualizar estado
                return rx.redirect("/hub")
            else:
                error_detail = res.json().get("detail", "Erro ao fazer login")
                self.error_message = error_detail
                logger.warning(f"Falha no login: {error_detail}")
    except Exception as e:
        self.error_message = f"Erro de conexão: {str(e)}"
        logger.error(f"Erro no login: {e}")
    finally:
        self.is_loading = False
```

```
# --- CHAT RAG ---
async def handle_chat_message(self):
    """Envia mensagem para RAG."""
    if not self.current_message.strip():
        return

    question = self.current_message
    self.chat_history.append((question, "Pensando..."))
    self.current_message = ""
    self.is_loading = True

    yield

    try:
        async with httpx.AsyncClient(timeout=60.0) as client:
            response = await client.post(
                f"{BACKEND_URL}/consultoria/chat",
                json={
                    "message": question,
                    "history": self.chat_history[:-1]
                },
                headers={"Authorization": f"Bearer {self.access_token}"}
            )

            if response.status_code == 200:
                answer = response.json().get("answer", "Sem resposta.")
                self.chat_history[-1] = (question, answer)
            else:
                self.chat_history[-1] = (question, "Erro ao processar.")
    except Exception as e:
        self.chat_history[-1] = (question, f"Erro: {str(e)}")
    finally:
        self.is_loading = False

# --- MÉTRICAS ---
async def fetch_metrics(self):
    """Busca métricas do backend."""
    self.loading_metrics = True
    try:
        async with httpx.AsyncClient(timeout=10.0) as client:
            response = await client.get(
                f"{BACKEND_URL}/metrics/latest",
                headers={"Authorization": f"Bearer {self.access_token}"}
            )

            if response.status_code == 200:
                data = response.json()
```

```
        self.temperatura = data.get("temperatura", 26.5)
        self.oxigenio = data.get("oxigenio", 7.2)
        self.ph = data.get("ph", 6.8)
        self.amonia = data.get("amonia", 0.5)
except Exception as e:
    logger.error(f"Erro ao buscar métricas: {e}")
finally:
    self.loading_metrics = False
```

frontend/ui/styles.py

```
import reflex as rx

# Paleta DrTilápia 1.3
PRIMARY = "#004e92"
ACCENT = "#ff7f50"
SECONDARY = "#00ced1"
BACKGROUND = "linear-gradient(135deg, #004e92 0%, #00ced1 100%)"
WHITE = "#ffffff"

# Glassmorphism
GLASSMORPHISM_BG = "rgba(255, 255, 255, 0.1)"
GLASSMORPHISM_BORDER = "1px solid rgba(255, 255, 255, 0.2)"

# Estilos de Componentes
NAV_STYLE = {
    "background": GLASSMORPHISM_BG,
    "backdrop_filter": "blur(12px)",
    "border": GLASSMORPHISM_BORDER,
}
```

frontend/ui/ui.py

```
import reflex as rx
from .pages.index import index
from .pages.login import login_page
from .pages.hub import hub
from .pages.consultoria import consultoria
from .pages.dashboard import dashboard
from .state import State

app = rx.App()

# Rotas Públicas
app.add_page(index, route="/")
app.add_page(login_page, route="/login")

# Rotas Protegidas: O evento check_login roda antes da renderização
app.add_page(hub, route="/hub", on_load=State.check_login)
app.add_page(consultoria, route="/consultoria", on_load=State.check_login)
app.add_page(dashboard, route="/dashboard", on_load=State.check_login)
```

frontend/ui/pages/admin_rag.py

```
import reflex as rx
from ..state import State
from ..styles import PRIMARY, ACCENT, SECONDARY, GLASSMORPHISM_BG,
GLASSMORPHISM_BORDER


def page_protegida(component: rx.Component) -> rx.Component:
    """Wrapper para proteção de rotas."""
    return rx.cond(
        State.is_authenticated,
        component,
        rx.redirect("/login")
    )


def admin_rag_content() -> rx.Component:
    """Conteúdo principal do painel admin RAG."""
    return rx.box(
        rx.vstack(
            # Header
            rx.hstack(
                rx.heading("Admin RAG - Gerenciar Documentos", size="9", color="white"),
                rx.spacer(),
                rx.button(
                    "Voltar",
                    on_click=rx.redirect("/hub"),
                    background=SECONDARY,
                    color="white",
                    _hover={"opacity": "0.8"}
                ),
                width="100%",
                padding="1em",
                background=PRIMARY
            ),

            # Conteúdo
            rx.vstack(
                rx.heading("Gerenciamento de Documentos", size="5", color="white"),

                rx.box(
                    rx.vstack(
                        rx.text("Funcionalidades disponíveis:", color="white", font_weight="bold"),
                        rx.unordered_list(
                            rx.list_item("Upload de novos documentos PDF"),
                            rx.list_item("Visualizar documentos indexados"),
```

```
            rx.list_item("Reindexar base de conhecimento"),
            rx.list_item("Gerenciar permissões de acesso"),
            color="white"
          ),
          spacing="1em"
        ),
        background="rgba(0,0,0,0.3)",
        padding="1.5em",
        border_radius="0.5em",
        width="100%"
      ),

      # Botões de ação
      rx.hstack(
        rx.button(
          "📤 Upload de Documento",
          background=ACCENT,
          color="white",
          padding="0.8em 1.5em",
          _hover={"opacity": "0.8"}
        ),
        rx.button(
          "🔄 Reindexar",
          background=SECONDARY,
          color="white",
          padding="0.8em 1.5em",
          _hover={"opacity": "0.8"}
        ),
        rx.button(
          "📋 Listar Documentos",
          background=PRIMARY,
          color="white",
          padding="0.8em 1.5em",
          _hover={"opacity": "0.8"}
        ),
        spacing="1em",
        width="100%",
        justify_content="center"
      ),

      spacing="1.5em",
      padding="2em",
      width="100%"
    ),

    spacing="1em",
    width="100%"
  ),
```

```
        width="100%",
        height="100vh",
        background="linear-gradient(135deg, #004e92 0%, #00ced1 100%)",
        padding="0"
    )


def admin_rag() -> rx.Component:
    """Página admin RAG com proteção de rotas (exportada como 'admin_rag' para ui.py)."""
    return page_protegida(admin_rag_content())
```

frontend/ui/pages/consultoria.py

```
import reflex as rx
from ..state import State
from ..styles import PRIMARY, ACCENT, SECONDARY, GLASSMORPHISM_BG,
GLASSMORPHISM_BORDER


def chat_message(message: tuple) -> rx.Component:
    """Renderiza uma mensagem individual do chat."""
    return rx.vstack(
        rx.box(
            rx.text(message[0], color="white", font_weight="bold"),
            background=PRIMARY,
            padding="1em",
            border_radius="1em 1em 0 1em",
            align_self="flex-end",
            max_width="80%",
        ),
        rx.box(
            rx.text(message[1], color="white"),
            background="rgba(255, 255, 255, 0.1)",
            padding="1em",
            border_radius="1em 1em 1em 0",
            align_self="flex-start",
            max_width="80%",
            backdrop_filter="blur(10px)",
            border=GLASSMORPHISM_BORDER,
        ),
        width="100%",
        spacing="2",
    )


def consultoria_content() -> rx.Component:
    """Conteúdo principal da Consultoria IA."""
    return rx.hstack(
        # Sidebar
        rx.vstack(
            rx.heading("Dr. Tilápia", size="6", color="white"),
            rx.divider(),
            rx.button(
                "Voltar ao Hub",
                on_click=rx.redirect("/hub"),
                variant="ghost",
                color_scheme="cyan",
                width="100%"
```

```
    ),
    width="250px",
    height="100vh",
    padding="2em",
    background="rgba(0, 0, 0, 0.4)",
    backdrop_filter="blur(15px)",
    border_right=GLASSMORPHISM_BORDER,
),

# Área de Chat
rx.vstack(
    rx.box(
        rx.heading(
            "Consultoria IA - Dr. Tilápia",
            size="7",  # CORRIGIDO: Escala 1-9 (7 é equivalente a large)
            color="white"
        ),
        padding="2em",
        width="100%",
        background="rgba(255, 255, 255, 0.05)",
        backdrop_filter="blur(5px)",
    ),

    # Histórico de Mensagens
    rx.scroll_area(
        rx.vstack(
            rx.foreach(State.chat_history, chat_message),
            width="100%",
            padding="2em",
            spacing="4",
        ),
        height="60vh",
        width="100%",
    ),

    # Input de Mensagem
    rx.hstack(
        rx.input(
            placeholder="Digite sua dúvida técnica...",
            value=State.current_message,
            on_change=State.set_current_message,
            width="100%",
            background="rgba(255, 255, 255, 0.1)",
            border=GLASSMORPHISM_BORDER,
            color="white",
        ),
        rx.button(
            "Enviar",
```

```
            on_click=State.handle_chat_message,
            background=ACCENT,
            color="white",
            is_loading=State.is_loading,
          ),
          padding="2em",
          width="100%",
          background="rgba(0, 0, 0, 0.2)",
        ),
        width="100%",
        height="100vh",
        spacing="0",
      ),
      width="100%",
      height="100vh",
      background="url('/hub01.jpeg')",
      background_size="cover",
      background_position="center",
      spacing="0",
    )


def consultoria() -> rx.Component:
    """Página de consultoria com proteção de rota via on_mount no ui.py."""
    return rx.fragment(
      rx.cond(
        State.is_authenticated,
        consultoria_content(),
        rx.center(rx.spinner(size="3"), height="100vh")
      ),
      on_mount=State.check_login  # Proteção correta (Evento, não Componente)
    )
```

frontend/ui/pages/dashboard.py

```
import reflex as rx
from ..state import State
from ..styles import PRIMARY, ACCENT, SECONDARY, GLASSMORPHISM_BG,
GLASSMORPHISM_BORDER


def metric_card(label: str, value: str, icon: str) -> rx.Component:
    """Card de métrica individual."""
    return rx.box(
        rx.vstack(
            rx.text(icon, font_size="2em"),
            rx.text(label, font_size="0.9em", color="rgba(255,255,255,0.7)"),
            rx.text(value, font_size="1.5em", font_weight="bold", color="white"),
            spacing="2",
            align_items="center",
            text_align="center",
        ),
        background=GLASSMORPHISM_BG,
        border=GLASSMORPHISM_BORDER,
        border_radius="1em",
        padding="1.5em",
        width="150px",
        backdrop_filter="blur(10px)",
        _hover={"transform": "scale(1.05)", "opacity": "0.95"},
    )


def dashboard_content() -> rx.Component:
    """Conteúdo do dashboard."""
    return rx.vstack(
        # Header
        rx.box(
            rx.vstack(
                rx.heading(
                    "Métricas do Sistema",
                    size="5",  # CORRIGIDO: "md" → "5"
                    color="white"
                ),
                rx.text(
                    "Monitoramento em tempo real do sistema Dr. Tilápia",
                    color="rgba(255,255,255,0.7)",
                    font_size="0.9em"
                ),
                spacing="2",
            ),
            padding="2em",
```

```
    background="rgba(255, 255, 255, 0.05)",
    backdrop_filter="blur(5px)",
    width="100%",
),

# Grid de Métricas
rx.grid(
    metric_card("Temperatura", State.temperatura, "🌡️"),
    metric_card("Oxigênio", State.oxigenio, "💨"),
    metric_card("pH", State.ph, "🧪"),
    metric_card("Amônia", State.amonia, "☠️"),
    columns="4",
    spacing="4",
    padding="2em",
    width="100%",
),

# Gráfico (placeholder)
rx.box(
    rx.text(
        "Gráfico de Tendências (Implementação Futura)",
        color="rgba(255,255,255,0.5)",
        text_align="center",
        padding="3em",
    ),
    background=GLASSMORPHISM_BG,
    border=GLASSMORPHISM_BORDER,
    border_radius="1em",
    margin="2em",
    width="calc(100% - 4em)",
    height="300px",
),

# Botão de Atualização
rx.button(
    "Atualizar Métricas",
    on_click=State.fetch_metrics,
    background=ACCENT,
    color="white",
    width="200px",
    margin="2em auto",
    is_loading=State.loading_metrics,
),

width="100%",
height="100vh",
background="url('/hub01.jpeg')",
```

```
        background_size="cover",
        background_position="center",
        spacing="0",
        overflow_y="auto",
    )


def page_protegida(component: rx.Component) -> rx.Component:
    """Wrapper de proteção de rota."""
    return rx.fragment(
        rx.cond(
            State.is_authenticated,
            component,
            rx.center(rx.spinner(size="3"), height="100vh")
        ),
        on_mount=State.check_login
    )


def dashboard() -> rx.Component:
    """Página de dashboard."""
    return page_protegida(dashboard_content())
```

frontend/ui/pages/hub.py

```
import reflex as rx
from ..state import State
from ..styles import PRIMARY, ACCENT, GLASSMORPHISM_BG,
GLASSMORPHISM_BORDER

def floating_card(title: str, description: str, href: str, icon: str, **kwargs) -> rx.Component:
  return rx.box(
    rx.link(
      rx.vstack(
        rx.text(icon, font_size="3em"),
        rx.text(title, font_size="1.3em", font_weight="bold", color="white"),
        rx.text(description, font_size="0.9em", color="rgba(255,255,255,0.8)"),
        spacing="2",
        align_items="center",
        text_align="center"
      ),
      href=href,
      _hover={"text_decoration": "none"}
    ),
    background=GLASSMORPHISM_BG,
    border=GLASSMORPHISM_BORDER,
    border_radius="1.5em",
    padding="2em",
    width="180px",
    height="180px",
    display="flex",
    align_items="center",
    justify_content="center",
    position="absolute",
    transition="all 0.3s ease-in-out",
    _hover={"transform": "scale(1.1)", "opacity": "0.95"},
    cursor="pointer",
    **kwargs
  )

def hub_content() -> rx.Component:
  return rx.box(
    rx.image(
      src="/hub01.jpeg", width="100%", height="100vh",
      object_fit="cover", position="absolute", z_index="-1", filter="brightness(0.6)"
    ),
    floating_card("Dashboard", "Métricas", "/dashboard", "📊", bottom="20%",
right="10%"),
    floating_card("Consultoria", "Chat IA", "/consultoria", "🤖", bottom="40%", left="10%"),
    floating_card("Admin", "RAG", "/admin_rag", "⚙️", top="20%", right="15%"),
    rx.box(
```

```
        rx.button("Sair", on_click=State.logout, background=ACCENT, color="white"),
        position="absolute", top="2em", right="2em", z_index="10"
      ),
      width="100%", height="100vh", position="relative", overflow="hidden"
    )

def hub() -> rx.Component:
    """Hub com proteção de rota e feedback visual."""
    return rx.fragment(
      rx.cond(
        State.is_authenticated,
        hub_content(),
        rx.center(
          rx.vstack(
            rx.spinner(size="3"),
            rx.text("Verificando autenticação...", color="white"),
            spacing="2",
            align_items="center"
          ),
          height="100vh",
          background="linear-gradient(135deg, #004e92 0%, #00ced1 100%)"
        )
      ),
      on_mount=State.check_login
    )
```

frontend/ui/pages/index.py

```
import reflex as rx
from ..styles import PRIMARY, ACCENT, BACKGROUND

def index():
    return rx.box(
        rx.box(
            rx.vstack(
                rx.heading("Dr. Tilápia 2.0", size="9", color="white"),
                rx.text("IA a serviço da sua piscicultura.", color="white", font_size="1.5em"),
                rx.link(
                    rx.button("Entrar no Sistema", bg=ACCENT, color="white", size="4"),
                    href="/login",
                ),
                spacing="5", align="center",
            ),
            background_image="url('/bg_hero.png')",
            background_size="cover",
            background_position="center",
            background_blend_mode="overlay",
            background_color="rgba(0, 8, 20, 0.6)",
            height="100vh", display="flex", align_items="center", justify_content="center",
width="100%",
        ),
        bg=BACKGROUND, width="100%",
    )
```

frontend/ui/pages/login.py

```
import reflex as rx
from ..state import State
from ..styles import PRIMARY, ACCENT, BACKGROUND, WHITE, SECONDARY

def login_page() -> rx.Component:
    return rx.center(
        rx.vstack(
            rx.heading("Dr. Tilápia 1.3", size="8", color=WHITE), # Size 1-9
            rx.input(placeholder="E-mail", on_change=State.set_user_email, width="100%"),
            rx.input(placeholder="Senha", type="password", on_change=State.set_password,
width="100%"),
            rx.button("Entrar", on_click=State.handle_login, width="100%", bg=ACCENT,
is_loading=State.is_loading),
            rx.cond(State.error_message != "", rx.text(State.error_message, color="red")),
            spacing="4",
            padding="2em",
            bg="rgba(255,255,255,0.1)",
            border_radius="15px",
            backdrop_filter="blur(10px)",
        ),
        height="100vh",
        bg=BACKGROUND,
    )
```

frontend/assets/bg_hero.png

frontend/assets/bg_hero_old.png

frontend/assets/favicon.ico

frontend/assets/Hub.jpeg

frontend/assets/hub01.jpeg

frontend/assets/ImagemPrincipal.png

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y build-essential libpq-dev && rm -rf
/var/lib/apt/lists/*
COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY backend/ .
EXPOSE 8080
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Estrutura_dir.py

```
from pathlib import Path

# Diretórios considerados "lixo" em projetos reais
IGNORAR_DIRS = {
    ".git",
    ".idea",
    ".vscode",
    "__pycache__",
    "node_modules",
    ".pytest_cache",
    "dist",
    "build",
    "venv",
    "env",
}

# Arquivos inúteis também podem ser ignorados aqui
IGNORAR_ARQUIVOS = {
    ".DS_Store",
    "Thumbs.db",
}


def gerar_arvore(diretorio, prefixo=""):
    caminho = Path(diretorio)

    # Lista somente itens relevantes
    itens = []
    for item in caminho.iterdir():

        # Ignorar ocultos (.qualquercoisa)
        if item.name.startswith(".") and item.name != ".gitignore":
            continue

        # Ignorar diretórios lixo
        if item.is_dir() and item.name in IGNORAR_DIRS:
            continue

        # Ignorar arquivos lixo
        if item.is_file() and item.name in IGNORAR_ARQUIVOS:
            continue

        itens.append(item)

    # Ordenação: pastas primeiro, depois arquivos
    itens.sort(key=lambda x: (not x.is_dir(), x.name.lower()))
```

```
    total = len(itens)
    for index, item in enumerate(itens):
        ultimo = (index == total - 1)
        conector = "└── " if ultimo else "├── "

        print(prefixo + conector + item.name)

        if item.is_dir():
            novo_prefixo = prefixo + ("    " if ultimo else "│   ")
            gerar_arvore(item, novo_prefixo)


# --- Execução ---
print("📦 Estrutura do Projeto (clean):")
gerar_arvore(".")
```

Supabase

.gitignore

```
# Byte-compiled / optimized / DLL files
__pycache__/
*.py[cod]
*$py.class

# Virtual Environments (MUITO IMPORTANTE)
env/
venv/
.env
.venv/

# Distribution / packaging
dist/
build/
*.egg-info/

# IDE specific settings (Opcional, mas recomendado)
.vscode/
.idea/
```